System Design - Modal Feature

Author: Nirmith Victor D'Almeida

Approach: Modal as a Screen

1. Overview
2. Scope & Non-Goals (rolling updates)
3. Data Flow
   - 3.1. APIs and data we want to use/ need to implement
   - 3.2. Data Input → Output Storage / Database Considerations
   - 3.3. Components Integration and Flow
   - 3.4. Implementation Details (update this to focus on screen as a modal but via a different action set instead on the go to screen open in)
   - 3.5. Timeline (will need to discuss tasks, flows and expectations with PM and designer for better predictability)
4. Architecture Design
   - 4.1. Current architecture (Before Modal Implementation)
   - 4.2. Anticipated architecture for Modal workflow (after Modal Implementation)
5. Breakpoints
   - Mobile
   - Tablet
   - Desktop
6. Sales Mgmt Use Case
7. Additional Considerations
   - 7.1. Non-Functional Requirements
   - 7.2. Metrics & Monitoring
   - 7.3. Testing/QA
   - 7.4. Known Limitations & Roadmap
   - 7.5. Open Questions/ Unknowns

1. Overview

TLDR: We're creating a Modal feature that lets users open mini-screens (like pop-up windows) on top of their current screen to do quick tasks, like entering data, without losing their place. This is a big deal for making our platform easier to use, especially for inventory tasks in Sprint 1. We'll set it up with an action set to configure a screen as a modal that can be triggered to appear on a button click, working separately from the main screen.

Goal: The Modal feature enables any existing screen in the no-code platform to be launched as an overlay dialog, allowing users to perform quick, contextual tasks (e.g., data entry, confirmations) without full-page navigation. For Sprint 1, we are focusing on a minimal setup using action set to display screens as modal overlays, with isolated state and lifecycle

management at runtime. Full user configuration options, such as tagging specific screens as modal in the Action Editor, are deferred to a later sprint to prioritize core functionality.

2. Scope & Non-Goals (rolling updates)

TLDR: For Sprint 1, we're focusing on the basics—getting pop-up windows to show up using a new set of actions, dimming the background so users focus on the pop-up, adding a simple close button (deffered for now), and updating the main screen when the pop-up closes after saving data.

In-scope (Sprint #1)	Out of scope	Status
Show Modal action launches overlay: Implement a new action set (e.g., "Show Modal" action) to launch a screen as a modal overlay via predefined or temporary configurations.	**Screen-to-Screen Navigation** Inside a Modal	PLANNED
Close Modal action set	**Nested Modals** (modal within modal) or wizards	PLANNED
Dimmed background & parent blocking	**Tabs or Signature Widgets** embedded in modals	PLANNED
Probably import Save and Close action set from classic and mimic behavior	**Full Mobile Responsiveness & Advanced Accessibility** (beyond basic scrolling/keyboard support)	PLANNED

Documentation & Diagrams	**Future Migration to a Native Modal Widget** (deprecation path for screen-as-modal)	PLANNED

3. Data Flow

TLDR: This part explains how information moves between the user, the system, and the pop-up windows. We're figuring out how a new set of actions starts a pop-up, how the system grabs the right info to show in it, and how data gets saved or updated when the pop-up is used. It's like mapping out a conversation between different parts of our app to make sure everything works smoothly.

3.1. APIs and data we want to use/ need to implement

TLDR: We're checking what tools (APIs) we already have to show pop-up content and what new ones we need to build. Right now, we've got a basic tool to get screen info, but we might need to tweak it. We also plan to add new tools to handle starting pop-ups with our new action set and saving data from them, even if some of that is for later sprints.

- APIs We Can Use Now:
  - **Screen Data API (Partially Fulfilled)**:
    - **Endpoint**: `GET /api/v1/screens/{screenId:guid}` in `ScreensController.cs` (`Runtime.Core.Api/Controllers/ScreensController.cs`).
    - **Purpose**: Retrieves screen metadata and controls for a given `screenId` using `ScreenLoadType.InfoAndControls`, which the frontend runtime (`Runtime.js`) can use to render a screen as a modal.
    - **Data Available**: JSON response with `IsSuccess: true`, `Data: _screenContext` (includes screen metadata and controls, but `Layout` and `Html` are set to `null`).
    - **Consideration**: Confirm if `ScreenLoadType.InfoAndControls` provides sufficient data for modal rendering. If not, investigate other load types or additional parameters to include full rendering data (like layout/HTML).
- APIs We might Need to Implement:
  - Action Execution API (For New Action Set):

    - Endpoint: Planned `POST /api/v1/actions/execute` in `ActionsController.cs` (`Runtime.Core.Api/Controllers/ActionsController.cs`).
    - Purpose: To support the new action set (e.g., "Show Modal" action) by retrieving predefined or test action configurations from storage and returning them as JSON responses with modal flags (e.g., `OpenIn: 3`) to the frontend for processing.
    - Consideration: Focus on minimal predefined configurations for Sprint 1 to enable rapid delivery of modal triggering via the new action set; full user configuration support deferred.
  - Modal State Update API (Future):
    - Endpoint: Planned `POST /api/v1/actions/stateupdate` in `ActionsController.cs`.
    - Purpose: To handle state changes (e.g., modal closed) and support parent component updates (FR-206) after save actions from the modal.
    - Consideration: Basic planning for Sprint 1 to ensure data updates; full implementation may be deferred post-MVP.
- APIs Outside Scope for Sprint of Runtime.Core.Api (For Reference):
  - **Action Configuration API**: Saves modal configurations from the action editor (AngularJS UI) to the backend. Likely handled by a separate backend service, not currently in `ActionsController.cs`. Data includes `ActionType: "GoToScreen"`, `ScreenId`, `OpenIn: 3`, and other modal options.

3.2. Data Input → Output Storage / Database Considerations

TLDR: This is about how data goes from a user's input in a pop-up to being saved in our system. We're making sure the behind-the-scenes storage can handle pop-up info triggered by our new action set without messing up other parts of the app, and planning how updates from a pop-up will refresh the main screen when it closes (for Sprint 1).

- **Input (User Configuration)**:
  - **Source**: User triggers a modal overlay using a new action set (e.g., "Show Modal" action) in the frontend runtime (`Runtime.js`), based on predefined or test configurations for Sprint 1. Full user configuration in the Action Editor (e.g., selecting "Open In: Modal") is deferred to a later sprint.

- **Data**: Trigger data includes a predefined `ActionType`, `ScreenId`, and modal flag (`OpenIn: 3`).
- **Storage Consideration**: For Sprint 1, predefined action configurations are hardcoded or stored minimally in the backend database via an external mechanism. Future user-driven configurations (deferred with FR-100) will be saved via an external Action Configuration API, requiring the database to support modal-specific fields without breaking existing action schemas.

- **Processing (Backend to Frontend Data Flow)**:
  - **Action Trigger**: User initiates the new action set in the frontend runtime (`Runtime.js`), which requests the action response from the backend via the planned `POST /api/v1/actions/execute` endpoint in `ActionsController.cs`.
  - **Backend Response**: The endpoint retrieves predefined action configurations from storage and returns them as JSON (e.g., `OpenIn: 3` for modal).
  - **Screen Data Request**: Frontend requests screen data using `GET /api/v1/screens/{screenId:guid}` from `ScreensController.cs`, fetching screen metadata and controls from a backend store.
  - **Storage Consideration**: The backend store for screen data (accessed via `IScreenContext` in `ScreensController.cs`) must ensure data is structured to support modal rendering triggered by the new action set (e.g., no assumptions of main screen context that conflict with state isolation). If additional data (like layout) is needed, the database schema or query logic may need adjustment to include it based on load type or parameters.
- **Output (Modal Rendering)**:
  - **Frontend Processing**: Frontend processes the action response (`runtimeActionProcessing.actions.js`), updates Redux state (`processNavigationFlow.js`), and renders the modal (`modal.js`) using screen data from `ScreensController.cs`.
  - **Data**: Output is the visual modal overlay with isolated state (`panelName`), displaying screen controls and metadata. On close after save, parent components update with modal data (FR-206 in scope for Sprint 1).
  - **Storage Consideration**: For future close functionality (beyond stub logic), user interactions (e.g., closing a modal) will send data back to the backend via the planned Modal State

Update API in `ActionsController.cs`. The database must support logging or storing state changes and triggering follow-up actions if configured, requiring fields for modal instance IDs, state changes, and user input to support parent updates (FR-206).

- **Database Design Notes**:
  - Ensure action configuration storage supports predefined modal flags (`OpenIn` as integer or enum) for the new action set in Sprint 1. (might not be necessary with the help of close action set)
  - Verify screen data storage allows flexible retrieval (e.g., different load types for controls vs. full layout) to adapt to modal rendering needs triggered by the new action set.
  - Plan for future modal state storage with tables or fields for lifecycle events (open, close) linked to action or screen IDs for tracking, analytics, and parent updates (FR-206).

3.3. Components Integration and Flow

TLDR: Here, we're showing how different pieces of our app work together to make pop-ups happen with a new set of actions. It's like a team effort—some parts handle user requests to start a pop-up, others manage the data, and some show it on screen. We're mapping who does what to avoid confusion.

- **Action Editor (AngularJS, Limited Role in Sprint 1)**:
  - **Role**: Provides a potential UI for configuring actions, but for Sprint 1, user-driven modal configuration is deferred (FR-100). The new action set (e.g., "Show Modal" action) will be triggered via predefined or test setups not reliant on user input in the Action Editor.
  - **Integration**: No direct integration for user configurations in Sprint 1; predefined configurations are managed externally or hardcoded for testing. Future integration (post-Sprint 1) will send configuration data to a backend API for storage.
  - **Flow**: Minimal user input → Predefined configurations used for new action set.
- **Backend (.NET, Runtime.Core.Api)**:
  - **Role**: Acts as an intermediary for delivering action responses and screen data to the frontend to support the new action set.
  - **Components**:
    - `ActionsController.cs`: Will host the planned `POST /api/v1/actions/execute` endpoint to return action responses with predefined modal flags (e.g., `OpenIn: 3`) for the new action set. Future role in handling state updates (`POST /api/v1/actions/stateupdate`) for parent component refresh (FR-206).

    - `ScreensController.cs`: Hosts `GET /api/v1/screens/{screenId:guid}` to provide screen metadata and controls for modal rendering initiated by the new action set.
  - **Integration**: Both controllers use injected services (`IActionContext`, `IScreenContext`) to access backend data stores. They integrate with the frontend runtime by providing JSON responses compatible with `runtimeActionProcessing.actions.js` and `Runtime.js`.
  - **Flow**: Frontend request for new action → API endpoint → Backend data retrieval → JSON response to frontend.
- **Frontend Runtime (React/Redux)**:
  - **Role**: Processes the new action set, manages state, and renders modals.
  - **Components**:
    - `runtimeActionProcessing.actions.js`: Processes action responses from the planned `ActionsController.cs` endpoint for the new action set (e.g., "Show Modal").
    - `processNavigationFlow.js`: Routes modal actions (e.g., `OpenIn: 3`) to `modalPanelLogic` for state updates.
    - `Runtime.js`: Observes Redux state and renders `modal.js` if a modal is visible as triggered by the new action set.
    - `modal.js`: Renders the modal screen with isolated state using data from `ScreensController.cs`.
  - **Integration**: Requests data from `Runtime.Core.Api` endpoints (`ActionsController.cs` for actions, `ScreensController.cs` for screen data). Uses Redux for state management to isolate modal instances initiated by the new action set.
  - **Flow**: API response for new action → Action processing → State update → Screen data request → Modal rendering.

3.4. Implementation Details (update this to focus on screen as a modal but via a different action set instead on the go to screen open in)

TLDR: This section outlines how we are implementing the modal feature using a new, dedicated action set to open customizable screens as modals. Unlike the classic GoToScreen flow, this new action set is tailored for modal overlays with isolated state and a paired close action. While

advanced user-driven configurations (e.g., via the Action Editor) are deferred, the modal will still render real screens based on dynamic inputs, not hardcoded content.

- **Current Implementation from POC**:
  - **Modal Infrastructure** (Deferred): The existing POC supports a "Screen in Modal" approach where the GoToScreen action allows "Open In: Modal" (`OpenIn: 3`) in the frontend, with Redux state, actions, and reducer for modal open implemented (per `progress.md`). This approach, which uses existing screens as modal content, is deferred to a later version to prioritize a simpler MVP for Sprint 1.
  - **Screen Data Delivery** (Reference for Future): `GET /api/v1/screens/{screenId:guid}` in `ScreensController.cs` (`Runtime.Core.Api/Controllers/ScreensController.cs`) returns screen metadata and controls via `_screenContext` with `ScreenLoadType.InfoAndControls` . Layout and Html are set to null, which may or may not be sufficient for modal rendering in the deferred "Screen in Modal" approach.
  - **Frontend Rendering** (Reference for Future): Modal rendering in the POC uses ReactModal in `modal.js` (`MethodUI/public/react/components/Runtime/views/modal.js`), integrated with `Runtime.js` (`MethodUI/public/react/components/Runtime/widgets/Runtime.js`), which observes modal state and instantiates a new Runtime for isolation. This is part of the deferred "Screen in Modal" implementation.
- **Revised Approach for Sprint 1 (New Action Set Focus)**:
  - For Sprint 1, we are sticking with the POC's "Screen in Modal" approach by creating and using a new action set (e.g., "Show Modal" action) for sprint 1. The goal is to deliver a minimal viable product (MVP) for inventory tasks without the complexity of integrating existing screens as modals, which is deferred to a later version. (https://methodme.slack.com/files/U08JRQQMP5E/F0926RS2H8R/image.png Connect your Slack account)
  - **Clarification:**
    - Rather than hardcoding modal content or showing placeholder UIs, Sprint 1 focuses on:
      - Delivering a **new "Show Modal" action set** that dynamically opens **real screens** in or as a modal.

    - Screens are passed via parameters (`screenId` , `params` , `openIn: 3`), **not hardcoded**.
    - A companion **"Close Modal" action** will dismiss the modal and optionally trigger parent updates (FR-206).
    - The goal is to **mimic classic behavior** using modern infrastructure, with basic UI polish and clean separation of concerns.

    We are **not** deferring “screen-as-modal” as a concept — we are simplifying the delivery pipeline by:

    - Avoiding Action Editor Go To Screen *Open In* tagging for now (FR-100 deferred).
    - Using direct backend inputs (action configurations or stubs) to simulate what Action Editor will eventually provide.
- **Gaps (Updated for Revised Approach)**:
    - Navigation issues within modals are out of scope for Sprint 1 and remain under investigation for future versions (e.g., in `processNavigationFlow.js` as part of the deferred "Screen in Modal" approach).
    - Missing Action Execution API in `ActionsController.cs` to deliver modal action responses; scope reduced to support predefined modal actions via the new action set rather than user-configured tagging (FR-100 deferred) or full screen integration.
    - User-driven modal tagging in Action Editor (FR-100) deferred; MVP will use new action set to mark screen or open a screen as a modal with suitable configurations based on customizations and preset (e.g., "Open screen as a Modal/Pop Up").
- **Key Files and Components** (From [memory-bank]() and `Runtime.Core.Api`):
    - **Action Configuration** (External, Limited): `m-one/apps/action-editor/src/angular/actions/goto-screen/tmpl.html` - UI for setting "Open In: Modal" is deferred for user configuration; Sprint 1 will rely on predefined setups for the new action set and is not dependent on this file for MVP.
    - **API Response Handling** (Frontend): `MethodUI/public/react/components/Runtime/state/runtimeActionProcessing.actions.js` - Will process backend action responses for the new action set (needs data from planned `ActionsController.cs` endpoint).
    - **Navigation/Modal Logic** (Frontend - for future): `MethodUI/public/react/components/Runtime/state/runtimeActionHandlers/processNavigationFlow.js` - Will handle modal routing for

actions from the new set with a simplified focus in Sprint 1: closing the modal and routing back to the home screen. Full "Screen in Modal" logic and broader navigation capabilities are deferred to a later version.

- **Runtime Rendering** (Frontend): `MethodUI/public/react/components/Runtime/widgets/Runtime.js` - Will observe modal state and render modals triggered by the new action set, using placeholder or simplified content for Sprint 1.
- **Modal Rendering** (Frontend): `MethodUI/public/react/components/Runtime/views/modal.js` - Will render the modal overlay with isolated state, focusing on basic UI or placeholder content rather than full screen integration for Sprint 1.
- **Screen Data API** (Backend, Optional for Sprint 1): `Runtime.Core.Api/Controllers/ScreensController.cs` - Provides `GetScreenAsync` for screen data; may be used minimally or bypassed for Sprint 1 if placeholder content is sufficient (full screen data integration deferred with "Screen in Modal" approach).
- **Action Execution API** (Backend, Planned): `Runtime.Core.Api/Controllers/ActionsController.cs` - Needs new endpoint (e.g., `POST /api/v1/actions/execute`) for action responses with modal flags, supporting the new action set with predefined configurations for Sprint 1.

- **Implementation Steps for Missing Components**:
  - **Action Execution API**:
    - **Location**: Add to `ActionsController.cs` in `Runtime.Core.Api/Controllers`.
    - **Task (Adjusted)**: Create a new endpoint (e.g., `Task<IActionResult> ExecuteActionAsync([FromBody] ActionRequestModel request)`) to retrieve predefined or test action configurations with modal flags (e.g., `OpenIn: 3` or equivalent) for the new action set (e.g., "Show Modal"). Focus on enabling basic modal triggering without reliance on full screen integration (deferred "Screen in Modal" approach). Full user configuration support also deferred (FR-100).
    - **Guidance for New Developer**: Follow the pattern of `CallServiceAsync` in `ActionsController.cs`. Use dependency injection for services, return `Ok`

with a success object, and handle errors similarly. Test with a simple action response to trigger a basic modal for Sprint 1 before considering complex screen data integration in later versions.

- **Confirm Screen Data Sufficiency**:
  - **Location**: Test `GetScreenAsync` in `ScreensController.cs` (optional for Sprint 1).
  - **Task (Adjusted)**: If screen data is needed for minimal modal content in Sprint 1, call the endpoint (`GET /api/v1/screens/{screenId:guid}`) with a valid `screenId` using a tool like Postman or a local debug environment to inspect the JSON response for sufficiency (controls, metadata). If placeholder or simplified content is used instead, this step can be deferred along with the full "Screen in Modal" approach. If Layout or full rendering data is needed later, investigate if `ScreenLoadType` can be parameterized or if a new endpoint variant is required.
  - **Guidance for New Developer**: Start by finding a valid `screenId` (ask a teammate or check test data) if screen data is required for Sprint 1. Use Postman or Visual Studio's API testing to make the call and document the response structure. If setup is tricky or placeholder content suffices, focus on existing API docs or sample responses and defer full testing to later versions.
- **Modal State Update API (Future)**:
  - **Location**: Plan for `ActionsController.cs` or a new controller in `Runtime.Core.Api`.
  - **Task (Adjusted)**: Design a future endpoint (e.g., `POST /api/v1/actions/stateupdate`) to accept state change notifications (e.g., modal closed) with a payload including `ModalInstanceId` and `StateChange`. Include basic support for FR-206 (update parent on close after save) in Sprint 1 planning, even with simplified modal content.
  - **Guidance for New Developer**: Defer full implementation until close functionality beyond stub logic is needed. For now, note the structure in documentation (mimic `CallServiceAsync` request/response pattern) to support parent updates (FR-206).

- **Coding Standards for a New Developer**:
  - Since you're new and unsure about standards, follow the existing patterns in `ScreensController.cs` and `ActionsController.cs`. Use async method

names with `Async` suffix (e.g., `ExecuteActionAsync`), inject dependencies in the constructor, and return `Ok` with a success object (e.g., `{ IsSuccess: true, Data: ... }`). Keep methods focused on one task. If a team style guide or senior input becomes available, adjust accordingly.

- Avoid complex changes initially. Start with testing existing endpoints (`GetScreenAsync`) if needed, before adding new ones for the action set. Document every step or finding to build knowledge and share with the team.

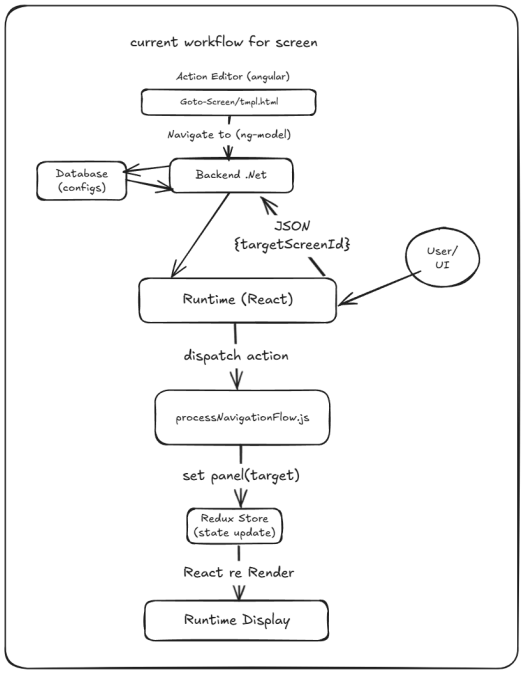
3.5. Timeline (will need to discuss tasks, flows and expectations with PM and designer for better predictability)

4. Architecture Design

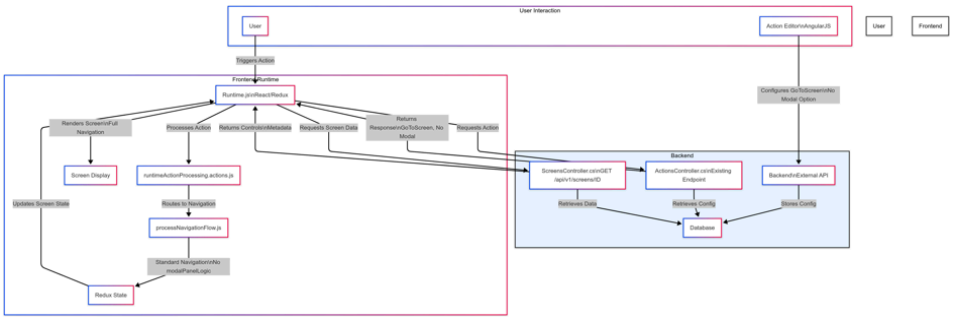
TLDR: This section shows how our app is built now (without pop-ups) and how it'll change to include them for Sprint 1. Think of it as a before-and-after blueprint—we're adding a way to show pop-ups on top of the main screen using a new set of actions, with just the basics for now, like dimming the background and updating the main screen when done saving data.

4.1. Current architecture (Before Modal Implementation)

In the legacy flow, all GoToScreen actions result in full-panel navigations via a single Runtime instance



current workflow for screen navigation (Nirmith view)



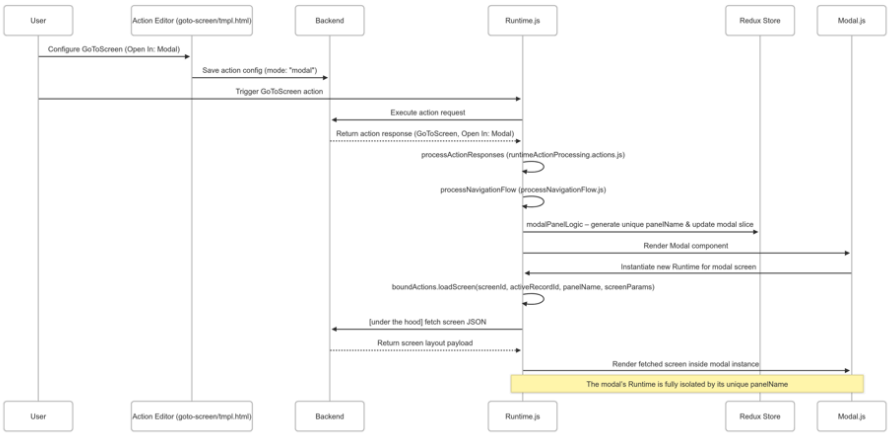
AI understanding of current flow

Description:

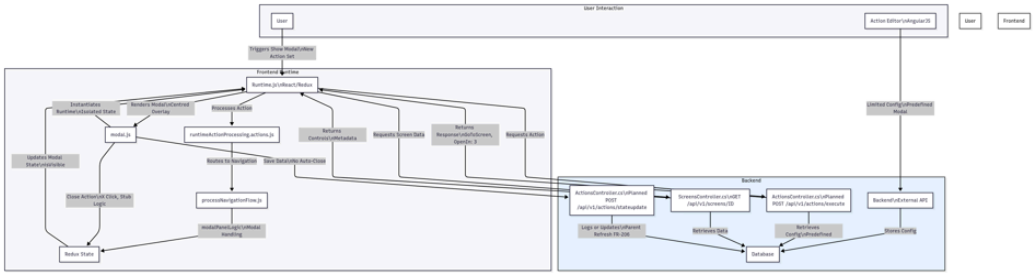
- **User Interaction**: Users configure actions in the AngularJS-based Action Editor and trigger them via the frontend.
- **Backend**: Configurations are stored in a database via an external API. The backend provides action responses (assumed via an existing endpoint in `ActionsController.cs`) and screen data (`ScreensController.cs`).
- **Frontend Runtime**: The runtime processes actions, routes them through standard navigation logic, updates Redux state, and renders the screen as a full navigation (not a modal).

4.2. Anticipated architecture for Modal workflow (after Modal Implementation)

With the Modal feature, GoToScreen open in will contain the option to mark “Open In: Modal” follow this augmented flow:



This is the anticipated flow for the modal (Help from Ben’s documentation and AI)



Description:

- **User Interaction**: Users configure actions with modal options in the Action Editor and trigger them via the frontend.
- **Backend**: Configurations are stored in a database. The backend provides action responses with modal flags via a planned endpoint in `ActionsController.cs`, screen data via `ScreensController.cs`, and future state update handling via another planned endpoint.
- **Frontend Runtime**: The runtime processes actions, routes them through modal-specific navigation logic, updates Redux state for modal visibility, and renders the modal as an overlay with isolated state.

5. Breakpoints

Mobile

Breakpoint Label	Device Type	Min Width	Min Height	Max Width	Max Height
SM	Mobile (Large)	417px	717px	—	—

Tablet

Breakpoint Label	Device Type	Min Width	Min Height	Max Width	Max Height
SM	Tablet (Small)	417px	372px	720px	980px
MD	Tablet (Medium)	616px	650px	720px	980px
LG	Tablet (Large)	673px	903px	720px	980px

Desktop

Breakpoint Label	Device Type	Min Width	Min Height	Max Width	Max Height

SM	Desktop (Small)	417px	372px	480px	420px
MD	Desktop (Medium)	698px	650px	720px	780px
LG	Desktop (Large)	975px	903px	960px	1020px

6. Sales Mgmt Use Case

Figma File

7. Additional Considerations

TLDR: This covers extra stuff we're thinking about beyond just building the pop-up. It includes how fast it should open, basic phone support, testing to make sure it works with our new action set, and questions we still need to answer, like whether clicking outside should close it. It's about making sure the feature is solid even if we're not doing everything yet.

7.1. Non-Functional Requirements

Performance: Modal open time ≤ 100 ms on typical payloads.

Responsiveness: Desktop and basic mobile support, including independent scrolling and keyboard adjustment.

7.2. Metrics & Monitoring

7.3. Testing/QA

Unit Tests: Cover `modalPanelLogic`, open/close reducer, and `Modal.js` behaviors.

Integration Tests: Simulate full workflow—from Action Editor config to runtime modal display.

E2E Scenarios: Verify blocking of background interactions, Escape-key close, and save-then-close flows.

7.4. Known Limitations & Roadmap

Navigation inside modals remains under investigation; future phases may enable back/forward within a modal context systemPatterns.

Modal-in-Modal and advanced mobile UX (e.g., device rotation handling) are deferred post-MVP.

7.5. Open Questions/ Unknowns

Should clicking outside the modal close it by default? - No

How should tabs and signature widgets render inside modals? - out of scope

What sizing guidelines (max-width, percentage) should apply across breakpoints? (refer System Design - Modal Feature | 5. Breakpoints)